

7.Process Synchronization and Child Process Monitoring Using waitpid()).

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

// Parse input into arguments
void parse_input(char *input, char **args) {
    char *token = strtok(input, " \t\n");
    int i = 0;
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " \t\n");
    }
    args[i] = NULL;
}

// Check if command exists in PATH
char *resolve_command(char *cmd) {
    char *path = getenv("PATH");
    char *copy = strdup(path);
    char *dir = strtok(copy, ":");
    static char fullpath[1024];
    struct stat sb;

    while (dir != NULL) {
        snprintf(fullpath, sizeof(fullpath), "%s/%s", dir, cmd);
        if (stat(fullpath, &sb) == 0 && sb.st_mode & S_IXUSR) {
            free(copy);
            return fullpath; // executable found
        }
        dir = strtok(NULL, ":");
    }
    free(copy);
    return NULL; // not found
}
```

```

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    pid_t pid;
    int status;

    while (1) {
        printf("myshell> ");
        if (fgets(input, MAX_INPUT, stdin) == NULL) break;

        parse_input(input, args);
        if (args[0] == NULL) continue;

        if (strcmp(args[0], "exit") == 0) break;

        // Resolve command
        char *cmd_path = resolve_command(args[0]);
        if (cmd_path == NULL) {
            printf("Command not found: %s\n", args[0]);
            continue;
        }

        pid = fork();
        if (pid == 0) {
            // Child executes
            execvp(cmd_path, args);
            perror("execvp failed");
            exit(EXIT_FAILURE);
        } else if (pid > 0) {
            // Parent monitors child
            if (waitpid(pid, &status, 0) == -1) {
                perror("waitpid failed");
            } else {
                if (WIFEXITED(status)) {
                    printf("Child exited with status %d\n", WEXITSTATUS(status));
                } else if (WIFSIGNALED(status)) {
                    printf("Child killed by signal %d\n", WTERMSIG(status));
                }
            }
        } else {
            perror("fork failed");
        }
    }
}

```

```

thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ nano skill7.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ gcc skill7.c -o skill7
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ ./skill7
myshell> ls -l
total 176
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16168 Aug 14 08:52 skill1
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 583 Aug 14 09:28 skill1.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16000 Aug 14 08:54 skill1b
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 188 Aug 14 08:53 skill1b.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16264 Aug 14 08:57 skill2
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 1605 Aug 14 08:56 skill2.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16264 Aug 29 08:18 skill3
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 1668 Aug 29 08:17 skill3.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16392 Sep 12 08:12 skill4
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 2748 Sep 12 08:12 skill4.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16560 Sep 21 07:01 skill5
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 1715 Sep 21 07:01 skill5.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16424 Sep 28 04:42 skill6
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 1253 Sep 28 04:42 skill6.c
-rwxr-xr-x 1 thumu_hemanth_kumar thumu_hemanth_kumar 16696 Sep 28 04:49 skill7
-rw-r--r-- 1 thumu_hemanth_kumar thumu_hemanth_kumar 2244 Sep 28 04:49 skill7.c
Child exited with status 0
myshell> echo Hello
Hello
Child exited with status 0
myshell>

```